

WEEK 8 PRACTICAL

1: Write a C program to demonstrate dynamic memory allocation using malloc(), calloc(), realloc(), and free(). Also identify and analyze memory leaks using Valgrind.

Aim : To understand dynamic memory allocation in C using malloc(), calloc(), realloc(), and free(), and to detect memory leaks using Valgrind.

Program

```
GNU nano 7.2          dynamic_memory.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p;

    // malloc
    p = malloc(2 * sizeof(int));
    p[0] = 10;
    p[1] = 20;

    printf("After malloc: %d %d\n", p[0], p[1]);

    // realloc
    p = realloc(p, 3 * sizeof(int));
    p[2] = 30;

    printf("After realloc: %d %d %d\n", p[0], p[1], p[2]);

    // free
    free(p);

    // calloc
    p = calloc(2, sizeof(int));

    printf("After calloc: %d %d\n", p[0], p[1]);

    // Intentionally not freeing p
    // This creates a memory leak

    return 0;
}
```

Output

```
alekhya@AlekhyaReddy:~/OSSP/Week8$ pwd
/home/sindhuja/OSSP/Week8
alekhya@AlekhyaReddy:~/OSSP/Week8$ nano dynamic_memory.c
alekhya@AlekhyaReddy:~/OSSP/Week8$ gcc dynamic_memory.c -o dynamic_memory
alekhya@AlekhyaReddy:~/OSSP/Week8$ ./dynamic_memory
After malloc: 10 20
After realloc: 10 20 30
After calloc: 0 0
alekhya@AlekhyaReddy:~/OSSP/Week8$ valgrind --leak-check=full ./dynamic_memory
==948== Memcheck, a memory error detector
==948== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==948== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==948== Command: ./dynamic_memory
After malloc: 10 20
After realloc: 10 20 30
After calloc: 0 0
==948==
==948== HEAP SUMMARY:
==948==   in use at exit: 8 bytes in 1 blocks
==948==   total heap usage: 4 allocs, 3 frees, 1,052 bytes allocated
==948==
==948== 8 bytes in 1 blocks are definitely lost in loss record 1 of 1
==948==    at 0x488D5B3: calloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==948==    by 0x10928E: main (in /home/sindhuja/OSSP/Week8/dynamic_memory)
==948==
==948== LEAK SUMMARY:
==948==    definitely lost: 8 bytes in 1 blocks
==948==    indirectly lost: 0 bytes in 0 blocks
==948==    possibly lost: 0 bytes in 0 blocks
==948==    still reachable: 0 bytes in 0 blocks
==948==    suppressed: 0 bytes in 0 blocks
==948==
==948== For lists of detected and suppressed errors, rerun with: -s
==948== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

2: Write a C program to demonstrate the Copy-on-Write (COW) mechanism after a `fork()` system call. Show that the parent and child processes have separate copies of the data when the child modifies it.

Aim : To demonstrate how **Copy-on-Write (COW)** works after `fork()`, and observe that the parent and child initially share memory pages. When the child modifies the data, the operating system creates a separate copy of the modified page for the child.

Program

```
GNU nano 7.2          copy_on_write.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int data = 10;

    printf("Before fork():\n");
    printf("Parent data: %d\n", data);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("\nChild process:\n");
        printf("Before modification: %d\n", data);

        data = 99;

        printf("After modification: %d\n", data);
    }
    else
    {
        wait(NULL);

        printf("\nParent process:\n");
        printf("Parent data: %d\n", data);
    }

    return 0;
}
```

Output :

```
alekhya@AlekhyaReddy:~/OSSP/Week8$ cd ~/OSSP/Week8
alekhya@AlekhyaReddy:~/OSSP/Week8$ nano copy_on_write.c
alekhya@AlekhyaReddy:~/OSSP/Week8$ gcc copy_on_write.c -o copy_on_write
alekhya@AlekhyaReddy:~/OSSP/Week8$ ./copy_on_write

Before fork():
Parent data: 10

Child process:
Before modification: 10
After modification: 99

Parent process:
Parent data: 10
alekhya@AlekhyaReddy:~/OSSP/Week8$
```